

dispatcher.cpp

[Go to the documentation of this file.](#)

```
1  #include "dispatcher.h"
2
3  // The thread constructor
4
5  Dispatcher::Dispatcher(int twait, QObject *parent)
6      : QThread(parent), abortQueue(false), dispatchCadence(twait)
7  {
8  }
9
10 // Add a message to the dispatch queue. For simplicity, we store it
11 // as "host msg ..." and then split it apart later as needed
12
13 void Dispatcher::addMessage(const QString &host, const QString &msg)
14 {
15     QMutexLocker locker(&mutex);
16     if (abortQueue) abortQueue = false;
17     msgQueue.enqueue(QString("%1 %2").arg(host,msg));
18     if (!isRunning()) start(); // restart the queue thread if inactive
19
20     emit queueChanged();
21 }
22
23 // The thread run() function
24
25 void Dispatcher::run()
26 {
27     QString msgStr;
28
29     forever {
30         mutex.lock();
31         if (msgQueue.isEmpty()) {
32             mutex.unlock();
33             break;
34         }
35         if (abortQueue) {
36             msgQueue.clear(); // clear the queue
37             mutex.unlock();
38             emit queueChanged();
39             emit error("Message Queue cleared and aborted");
40             break;
41         }
42
43         // Get the next message in the queue
44
45         msgStr = msgQueue.dequeue();
46         currentMsg = msgStr;
47         emit queueChanged();
48         mutex.unlock();
49
50         // Send the message then pause for the requested time
51
52         if (!msgStr.isEmpty()) {
53             QString hostAddr = msgStr.section(" ",0,0);
54             QString msgBody = msgStr.section(" ",1,-1);
55             emit dispatch(hostAddr,msgBody);
56             if (dispatchCadence > 0)
57                 msleep((unsigned long)(dispatchCadence));
58         }
59     }
60
61 }
62
63 // Return the number of pending message in the dispatch queue
64
65 int Dispatcher::numPending()
66 {
67     QMutexLocker locker(&mutex);
68     return msgQueue.size();
69 }
70
71 // Return the current message (last message dispatched)
72
```

```

73 QString Dispatcher::current()
74 {
75     QMutexLocker locker(&mutex);
76     return currentMsg;
77 }
78
79 // Abort the message queue. Implemented as a public slot so we can
80 // bind this slot to signals from, for example, an queue abort button
81 // in the parent application.
82
83 void Dispatcher::abort()
84 {
85     QMutexLocker locker(&mutex);
86     abortQueue = true;
87 }
88
89 // Set the dispatch cadence (time between queued message dispatches)
90 // in milliseconds
91
92 void Dispatcher::setCadence(const int &msec)
93 {
94     QMutexLocker locker(&mutex);
95     if (msec > 0) dispatchCadence = msec;
96 }
97
98 // Return the dispatch cadence in milliseconds
99
100 int Dispatcher::cadence()
101 {
102     QMutexLocker locker(&mutex);
103     return dispatchCadence;
104 }
105
106 // Return the current contents of the dispatch queue
107
108 QStringList Dispatcher::queueList()
109 {
110     QMutexLocker locker(&mutex);
111     QStringList queue;
112     queue.clear();
113     if (msgQueue.isEmpty()) return queue;
114     for (int i=0;i<msgQueue.size();i++)
115         queue.insert(i,msgQueue.at(i));
116     return queue;
117 }
118

```